VFS eVTOL Drone
("*Cloudy with a Chance of Medical Supplies*")
Critical Design Review

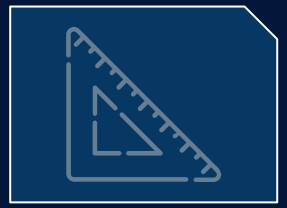
Adarsh Sheth, Zixiao Zhang,
Aaron Esquivel, Juan Polanco

Sensor Suite/DSS

Data Requirements

Global Position

Velocity & Angle
Acceleration



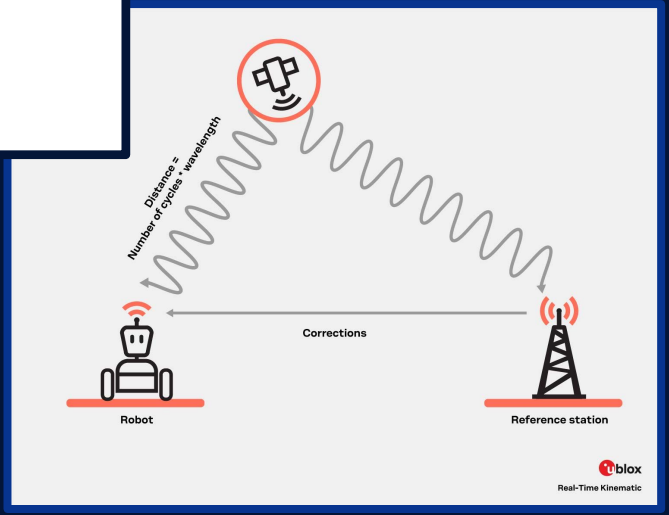
Distance from ground

Terrain
Recognition

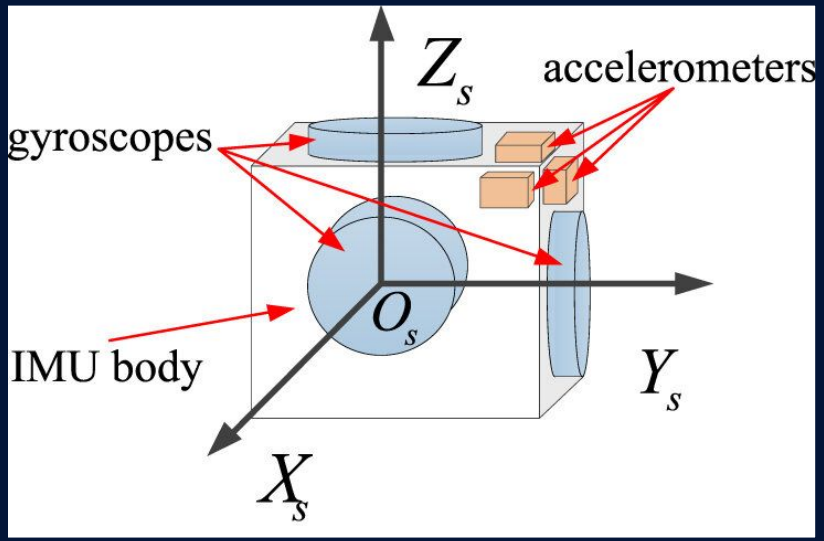
Obstacle
Recognition

Criteria	Weight	IMU	Long Range LIDAR	RTK GNSS	Ultrasonic	mmWave FMCW Radar	UWB Radar	Infrared	SAR Radar	Altimeter
Power Use	30	4	3	3	4	2	2	4	1	2
Cost	5	4	2	3	4	4	4	1	2	2
Range	15	4	4	4	1	3	2	4	4	4
Weight	20	4	3	2	2	2	1	4	2	3
Effectiveness	40	3	2	4	1	3	2	4	4	4
Totals		364	273	331	214	259	191	386	273	318

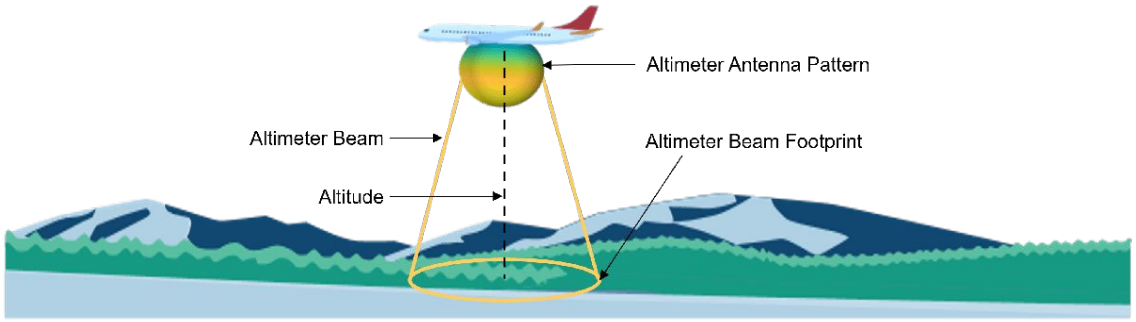
RTK GNSS location sensor



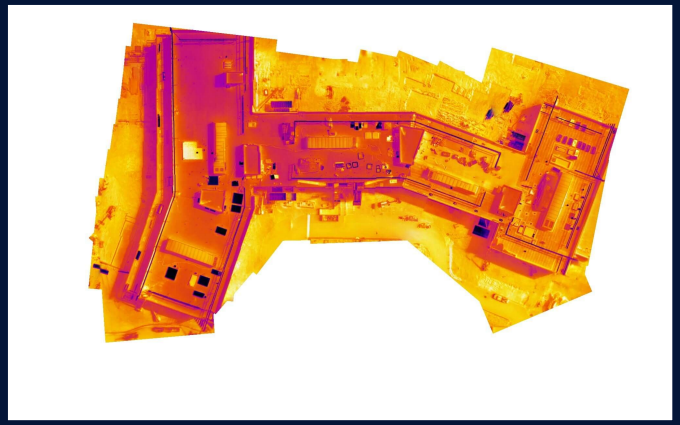
IMU (Inertial Measurement Unit)



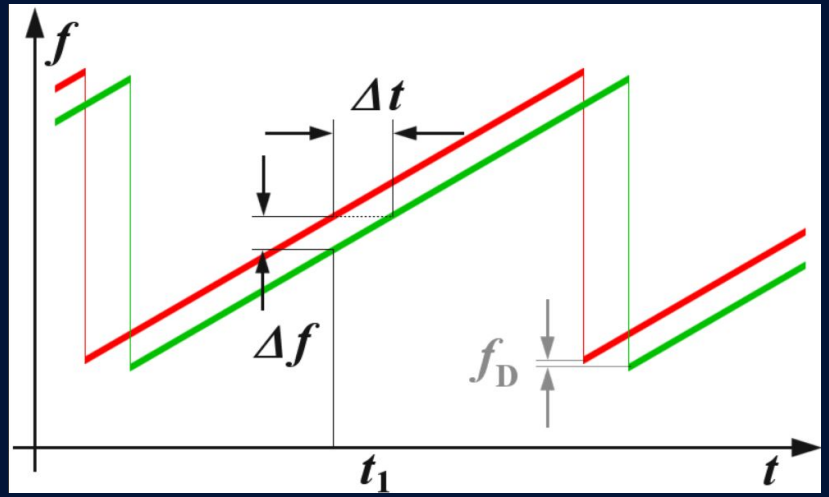
Radar Altimeter

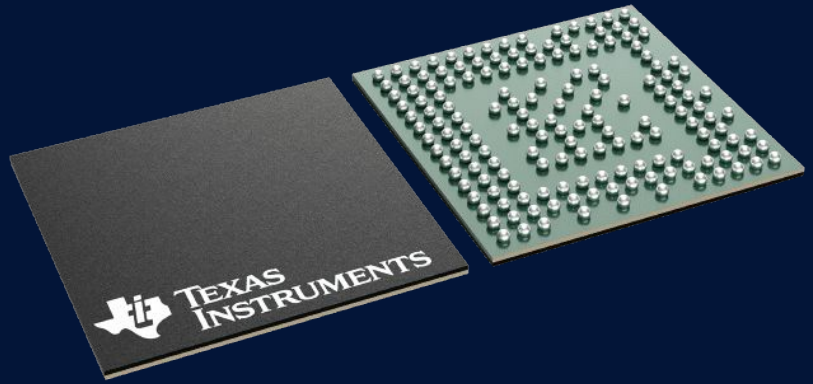


Dual Radiometric Thermal & Visible Module



mmWave FMCW Radar





Design Configuration

The Mission

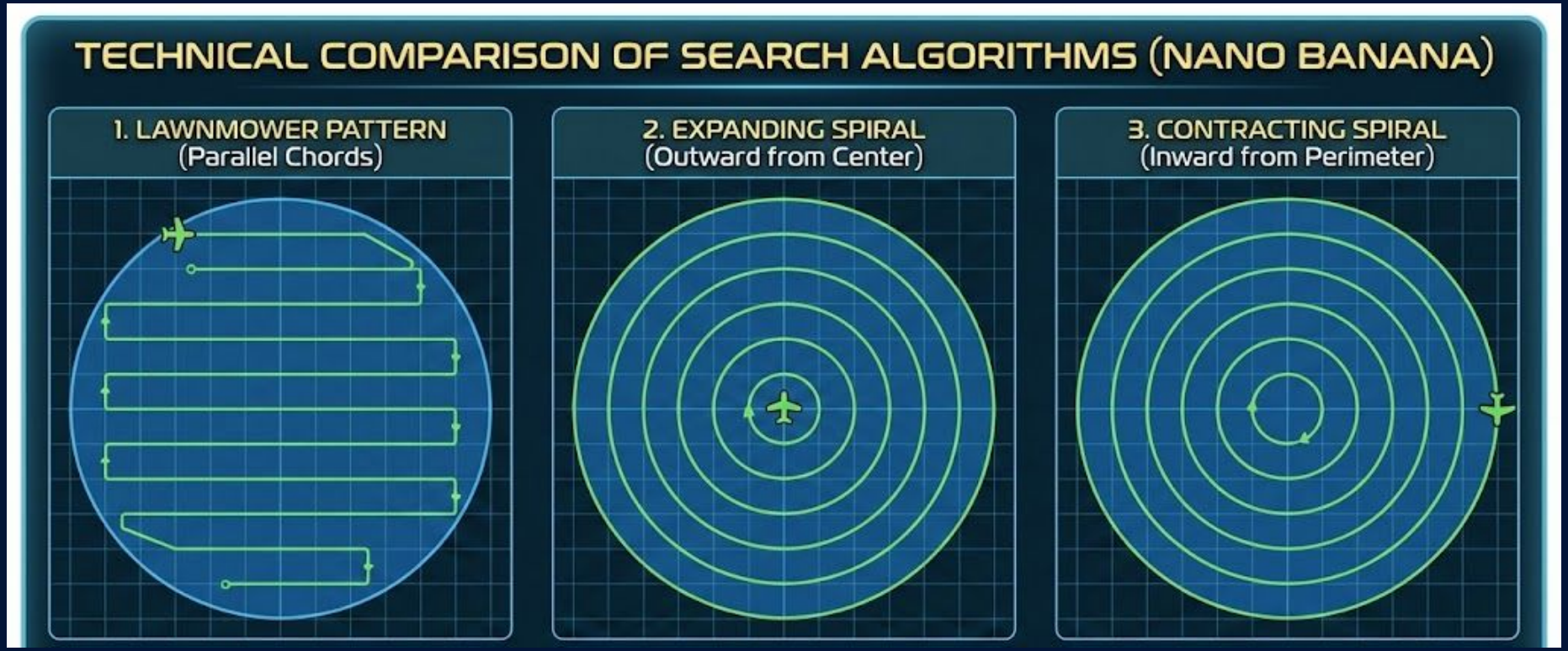
- Stability
- Hover Power
- Endurance
- Cruise Speed
- Stall Speed
- Mechanical Complexity/ Manufacturability
- Payload delivery accuracy

Final Four Design Matrix

Parameter	Weight	Lift + Cruise ✓	Tiltrotors ✗	Quad Bi-Plane ✗	Tiltwing ✗
Stability (Control Authority and Dynamic Response)	30	3	2	4	1
Hover Power	25	4	2	3	1
Endurance (Flight Time)	10	4	1	2	3
Stall Speed	10	3	1	4	2
Mechanical Complexity / Manufacturability	10	4	3	2	1
Payload Delivery Accuracy	10	4	3	2	1
Cruise Speed	5	2	4	1	3
Total	100	350	210	300	140

Hiker search Algorithm

Search Algorithm Methodology



Search Algorithm Methodology

```python
# -- CONSTANTS --
searchDiameter = 1*1609.344*2                    # [m] 1 mile radius area diameter
searchArea = math.pi*(searchDiameter/2)**2       # [m^2] total search area (circular)
scanningWidth = 840*math.sin(32*2*math.pi/180)  # [m] swath width at 840m AGL & 32° FOV
velocity = 8.94                                  # [m/s] aircraft ground speed; limited by the infared sensor
nTrials = 2000                                   # Number of random points to test for average time
```

```python
# --------- 1. LAWNMOWER SEARCH PATTERN ---------
def lawnmower_search_time(searchDiameter, searchArea, scanWidth, velocity, nTrials):
    """
    Lawnmower pattern optimized for circular search area.

    Pattern: Parallel strips running across the circular area. Each strip is a
    straight line (chord) that intersects the circle. Strips near the center are
    longer (approaching full diameter), while strips near the edges are shorter.
    The aircraft flies each chord, then makes a perpendicular turn to the next
    parallel chord, spaced by the scanning width apart.

    Shape covered: Series of parallel rectangular strips that collectively
    approximate the circular search area. Some strip area extends beyond the
    circle at the edges, creating slight overlap waste.
    """
    radius = searchDiameter / 2.0

    # Number of strips required (round up for full coverage)
    nStrips = math.ceil(searchDiameter / scanWidth)

    # Calculate max distance and coverage
    maxDistance = 0
    coveredArea = 0
    stripInfo = []  # Store strip information for time-to-target calculation

    for i in range(nStrips):
        # Distance from center to strip centerline
        stripOffset = -radius + (i + 0.5) * scanWidth

        # Check if strip intersects circle
        if abs(stripOffset) < radius:
            # Chord length at this offset using: chord = 2*sqrt(r^2 - d^2)
            chordLength = 2 * math.sqrt(radius**2 - stripOffset**2)
            maxDistance += chordLength

            # Store strip info: (stripNumber, yPosition, chordLength, cumulativeDistance)
            stripInfo.append((i, stripOffset, chordLength, maxDistance))

            # Area covered by this strip (rectangular approximation)
            coveredArea += chordLength * scanWidth

        # Add turn distance if not the last strip
        if i < nStrips - 1:
            maxDistance += scanWidth

    # Max time to cover entire area
    maxTime = maxDistance / velocity

    # Calculate average time to reach random targets
    timeToTargets = []

    for trial in range(nTrials):
        targetX, targetY = generate_random_point_in_circle(radius)

        # Find which strip contains this target
        # Strips run horizontally (parallel to x-axis), so check y-coordinate
        distanceToTarget = 0
        found = False

        for i, (stripNum, yPos, chordLen, cumDist) in enumerate(stripInfo):
            # Check if target's y-coordinate falls within this strip's swath
            if abs(targetY - yPos) <= scanWidth / 2:
                # Target is in this strip
                # Distance = all previous strips + distance along this strip to target
                if i == 0:
                    distanceToTarget = cumDist - chordLen + abs(targetX + chordLen/2)
                else:
                    prevCumDist = stripInfo[i-1][3]
                    distanceToTarget = prevCumDist + scanWidth + abs(targetX + chordLen/2)
                found = True
                break

        if found:
            timeToTargets.append(distanceToTarget / velocity)

    avgTimeToTarget = sum(timeToTargets) / len(timeToTargets) if timeToTargets else maxTime

    # Coverage metrics
    overagePercent = ((coveredArea - searchArea) / searchArea) * 100

    print("\n" + "="*60)
    print("LAWNMOWER PATTERN (Circular Optimized)")
    print("="*60)
    print("Pattern shape: Parallel chords across circle")
    print("Number of strips needed: {}".format(nStrips))
    print("Total flight distance: {:.1f} m".format(maxDistance))
    print("Max search time (full area): {:.1f} s ({:.1f} min)".format(maxTime, maxTime/60))
    print("Avg time to random target: {:.1f} s ({:.1f} min)".format(avgTimeToTarget, avgTimeToTarget/60))
    print("Target area: {:.1f} m^2".format(searchArea))
    print("Covered area: {:.1f} m^2".format(coveredArea))
    print("Area overage: {:.2f}% (excess coverage beyond target)".format(overagePercent))

    return nStrips, maxDistance, maxTime, avgTimeToTarget
```

```python
# --------- 2. EXPANDING SPIRAL PATTERN ---------
def spiral_search_time(searchDiameter, searchArea, swathWidth, velocity, nTrials):
    """
    Expanding Archimedean spiral starting from center and spiraling outward.

    Pattern: Continuous spiral path starting at the center point and expanding
    outward in a smooth curve. Each complete revolution increases the radius by
    the scanning width, ensuring full coverage with no gaps. The spiral continues
    until reaching the outer edge of the circular search area.

    Shape covered: Smooth circular coverage that naturally matches the circular
    search area. Minimal wasted coverage since the spiral follows the natural
    geometry of the circle.
    """
    areaRadius = searchDiameter / 2.0

    # Each loop outwards: radius increases by swathWidth
    nLoops = math.ceil(areaRadius / swathWidth)

    # Calculate total distance by summing circumferences
    maxDistance = 0
    coveredArea = 0

    for i in range(nLoops):
        radius_i = swathWidth * (i + 1)

        # Don't exceed the search area
        if radius_i > areaRadius:
            radius_i = areaRadius
            arcLength = 2 * math.pi * radius_i
            maxDistance += arcLength
            coveredArea = math.pi * areaRadius**2
            break
        else:
            arcLength = 2 * math.pi * radius_i
            maxDistance += arcLength
            coveredArea = math.pi * radius_i**2

    maxTime = maxDistance / velocity

    # Calculate average time to reach random targets
    timeToTargets = []

    for trial in range(nTrials):
        targetX, targetY = generate_random_point_in_circle(areaRadius)
        targetRadius = math.sqrt(targetX**2 + targetY**2)

        # Find which loop the target is in
        loopNum = math.ceil(targetRadius / swathWidth)
        if loopNum < 1:
            loopNum = 1

        # Distance = sum of all loops up to and including target loop
        distanceToTarget = 0
        for i in range(loopNum):
            r = swathWidth * (i + 1)
            if r > areaRadius:
                r = areaRadius
            distanceToTarget += 2 * math.pi * r

        # Approximate partial arc within final loop
        # Average case: target is halfway through the loop
        finalLoopRadius = min(swathWidth * loopNum, areaRadius)
        distanceToTarget -= math.pi * finalLoopRadius  # Subtract half loop (avg case)

        timeToTargets.append(distanceToTarget / velocity)

    avgTimeToTarget = sum(timeToTargets) / len(timeToTargets)

    # Coverage metrics
    overagePercent = ((coveredArea - searchArea) / searchArea) * 100

    print("\n" + "="*60)
    print("OUTWARD SPIRAL PATTERN (From Center)")
    print("="*60)
    print("Pattern shape: Archimedean spiral expanding outward")
    print("Number of spiral loops: {}".format(nLoops))
    print("Total flight distance: {:.1f} m".format(maxDistance))
    print("Max search time (full area): {:.1f} s ({:.1f} min)".format(maxTime, maxTime/60))
    print("Avg time to random target: {:.1f} s ({:.1f} min)".format(avgTimeToTarget, avgTimeToTarget/60))
    print("Target area: {:.1f} m^2".format(searchArea))
    print("Covered area: {:.1f} m^2".format(coveredArea))
    print("Area overage: {:.2f}% (excess coverage beyond target)".format(overagePercent))

    return nLoops, maxDistance, maxTime, avgTimeToTarget
```

```python
# --------- 3. INWARD SPIRAL PATTERN ---------
def inward_spiral_search_time(searchDiameter, searchArea, swathWidth, velocity, nTrials):
    """
    Contracting Archimedean spiral starting from perimeter and spiraling inward.

    Pattern: Continuous spiral path starting at the outer edge of the search area
    and spiraling inward toward the center. Each complete revolution decreases the
    radius by the scanning width. The spiral continues until reaching the center
    point of the circular search area.

    Shape covered: Smooth circular coverage identical to outward spiral but with
    reversed direction. Naturally matches the circular search area with minimal
    wasted coverage.
    """
    areaRadius = searchDiameter / 2.0

    # Each loop inward: radius decreases by swathWidth
    nLoops = math.ceil(areaRadius / swathWidth)

    # Calculate total distance by summing circumferences from outside to inside
    maxDistance = 0
    coveredArea = searchArea  # Spiral covers the full circular area

    for i in range(nLoops):
        radius_i = areaRadius - (swathWidth * i)

        if radius_i <= 0:
            break

        arcLength = 2 * math.pi * radius_i
        maxDistance += arcLength

    maxTime = maxDistance / velocity

    # Calculate average time to reach random targets
    timeToTargets = []

    for trial in range(nTrials):
        targetX, targetY = generate_random_point_in_circle(areaRadius)
        targetRadius = math.sqrt(targetX**2 + targetY**2)

        # For inward spiral, we reach outer targets first
        # Distance = sum of loops from outside until we reach target radius
        distanceToTarget = 0
        currentRadius = areaRadius

        for i in range(nLoops):
            r = areaRadius - (swathWidth * i)
            if r <= 0:
                break

            if r >= targetRadius:
                distanceToTarget += 2 * math.pi * r
                currentRadius = r
            else:
                # Target is between this loop and previous
                # Add partial arc (average: half loop)
                distanceToTarget += math.pi * currentRadius
                break

        timeToTargets.append(distanceToTarget / velocity)

    avgTimeToTarget = sum(timeToTargets) / len(timeToTargets)

    # Coverage metrics
    overagePercent = ((coveredArea - searchArea) / searchArea) * 100

    print("\n" + "="*60)
    print("INWARD SPIRAL PATTERN (From Perimeter)")
    print("="*60)
    print("Pattern shape: Archimedean spiral contracting inward")
    print("Number of spiral loops: {}".format(nLoops))
    print("Total flight distance: {:.1f} m".format(maxDistance))
    print("Max search time (full area): {:.1f} s ({:.1f} min)".format(maxTime, maxTime/60))
    print("Avg time to random target: {:.1f} s ({:.1f} min)".format(avgTimeToTarget, avgTimeToTarget/60))
    print("Target area: {:.1f} m^2".format(searchArea))
    print("Covered area: {:.1f} m^2".format(coveredArea))
    print("Area overage: {:.2f}% (excess coverage beyond target)".format(overagePercent))

    return nLoops, maxDistance, maxTime, avgTimeToTarget

# -- Run all three patterns --
lm_strips, lm_dist, lm_maxtime, lm_avgtime = lawnmower_search_time(searchDiameter, searchArea, scanningWidth, velocity, nTrials)
sp_loops, sp_dist, sp_maxtime, sp_avgtime = spiral_search_time(searchDiameter, searchArea, scanningWidth, velocity, nTrials)
in_loops, in_dist, in_maxtime, in_avgtime = inward_spiral_search_time(searchDiameter, searchArea, scanningWidth, velocity, nTrials)

# -- Summary comparison --
print("\n" + "="*60)
print("COMPARISON SUMMARY")
print("="*60)
print("Pattern          | Distance (m) | Max Time (min) | Avg Time (min)")
print("-"*60)
print("Lawnmower        | {:>11.1f}  | {:>13.1f}  | {:>13.1f}".format(lm_dist, lm_maxtime/60, lm_avgtime/60)
print("Outward Spiral   | {:>11.1f}  | {:>13.1f}  | {:>13.1f}".format(sp_dist, sp_maxtime/60, sp_avgtime/60)
print("Inward Spiral    | {:>11.1f}  | {:>13.1f}  | {:>13.1f}".format(in_dist, in_maxtime/60, in_avgtime/60)
print("="*60)
print("\nAverage Time Comparison (vs Lawnmower):")
print("Outward Spiral: {:+.1f}%".format((sp_avgtime-lm_avgtime)/lm_avgtime*100)
print("Inward Spiral: {:+.1f}%".format((in_avgtime-lm_avgtime)/lm_avgtime*100)
```

Final Iteration Search Flight Results

```
# -- CONSTANTS --
searchDiameter = 1*1609.344*2                    # [m] 1 mile radius area diameter
searchArea = math.pi*(searchDiameter/2)**2       # [m^2] total search area (circular)
scanningWidth = 840*math.sin(32*2*math.pi/180)  # [m] swath width at 840m AGL & 32° FOV
velocity = 8.94                                 # [m/s] aircraft ground speed; limited by the infared sensor
nTrials = 2000                                   # Number of random points to test for average time
```

```
================================================================
COMPARISON SUMMARY
================================================================
Pattern          | Distance (m) | Max Time (min) | Avg Time (min)
----------------------------------------------------------------
Lawnmower        |     13803.5  |          25.7  |          12.3
Outward Spiral   |     24343.0  |          45.4  |          17.3
Inward Spiral    |     16104.3  |          30.0  |          29.9
================================================================
```

Wing Sizing

Wing Sizing - Simulating Configurations

```python
def evaluate_designs(MTOW_range, S_range, A_range,hover_frac_search_range):
    """
    Loops through all sizings, assembles dicts for feasible designs.
    """
    results = []
    for hover_frac_search in frange(*hover_frac_search_range):
        for W_cruise in frange(*MTOW_range):
            for S_cruise in frange(*S_range):
                for A_hover in frange(*A_range):

                    # Geometry
                    sigma_cruise, sigma_hover, b_cruise, D_rotors_hover = wing_and_rotor_geometry(W_cruise, S_cruise, A_hover)
                    if b_cruise <= 1.1 * D_rotors_hover:
                        continue

                    # Aerodynamics
                    C_L_cruise, AOA_cruise, C_D_cruise = aerodynamic_coeffs(W_cruise, S_cruise, b_cruise,hover_frac_search)
                    if C_L_cruise is None:
                        continue

                    # Power/Energy
                    D_cruise, P_induced_hover, P_cruise, P_hover, P_ascent, P_descent, E_mission_Wh = power_and_energy(W_cruise, S_cruise, A_hover, C_D_cruise,hover_frac_search)

                    # Battery
                    battery_mass_batt, E_usable_batt, ratio_batt = battery_sizing(W_cruise, E_mission_Wh)

                    # Stall speed and conversions
                    V_stall_cruise = stall_speed(W_cruise, S_cruise,hover_frac_search)
                    W_lbs_cruise = W_cruise / 4.44822
                    if (FoS*V_stall_cruise>v_search):
                        continue

                    # Record if feasible
                    if ratio_batt >= 100:
                        results.append({
                            "W_N_cruise": round(W_cruise, 2),
                            "W_lbs_cruise": round(W_lbs_cruise,2),
                            "sigma_cruise": round(sigma_cruise,2),
                            "sigma_hover": round(sigma_hover,2),
                            "S_cruise": round(S_cruise,2),
                            "A_hover": round(A_hover,2),
                            "b_cruise": round(b_cruise,2),
                            "D_rotors_hover": round(D_rotors_hover,2),
                            "V_stall_cruise": round(V_stall_cruise,2),
                            "E_usable_batt": round(E_usable_batt,2),
                            "E_mission_Wh": round(E_mission_Wh,2),
                            "AOA_cruise": round(AOA_cruise,2),
                            "C_L_cruise": round(C_L_cruise,2),
                            "C_D_cruise": round(C_D_cruise,4),
                            "LD_cruise": round(C_L_cruise/C_D_cruise,2),
                            "D_cruise": round(D_cruise,2),
                            "ratio_batt": round(ratio_batt,2),
                            "hover_frac_search": round(hover_frac_search,2)
                        })
    return results
```

The following range determines the breadth of the configuations calculated and step size defines the fidelity of the simulation.

```python
##########################################################################
# 4. Vehicle Sizing Ranges

MTOW_range = (25, 500, 1)              # [W_cruise]: N
S_range = (1, 30.0, 0.1)               # [S_cruise]: m^2
A_range = (1, 30.0, 0.1)               # [A_hover]: m^2
hover_frac_search_range = (0,1,0.01)   # [hover_frac_search]: unitless
    # the percent of max RPM that the lift rotors spin at during CRUISE in SEARCH
```

476 MTOW × 291 S × 291 A × 101 Hover Fraction results in 4,071,123,756 total simulation iterations

To sort these designs, a weighted scoring system is developed and defined in the following code,

Wing Sizing - Constraints, pt. 1

(1) $V_{stall}(FoS) < V_{cruise}$

where $V_{stall} = \sqrt{\frac{MTOW * hover_{fraction}}{qSC_{L,max}}}$, FoS = 1.1, $V_{cruise} = 8.94\ m/s$,

in which V_{cruise} is a constraint derived from frame of the thermal sensor.

```
def stall_speed(W_cruise, S_cruise,hover_frac_search):
    """
    Returns stall speed [m/s] for current config.
    """
    W_felt = W_cruise*(1-hover_frac_search)
    V_stall_cruise = math.sqrt((2 * W_felt) / (rho_cruise * S_cruise * C_L_max_cruise))
    # print(V_stall_cruise)
    return V_stall_cruise
```

(2) $D_{rotor,combinated} + x_{props} < S_{wing} - S_{fuselage}$

where $D_{rotor,combinated} = 2 * \sqrt{\frac{A_{hover}}{4\pi}}$, non-rotor distances on the wingspan are

$x_{props} + S_{fuselage} \approx 1.1 * D_{rotor,combinated}$,

in which A and S are simulation-iterated values

(3) $E_{usable} / E_{mission} \geq 100\%$

where E_{usable} is first calculated,

Wing Sizing - Constraints, pt. 2

```python
def battery_sizing(W_cruise, E_mission_Wh):
    """
    Calculates battery mass/usable Wh/energy ratio.
    """
    specific_energy_batt = 65 * 2.205          # Wh/kg
    battery_mass_batt = 0.3 * ((W_cruise - 22.24 - 0.381*9.81)/9.81)    # only place where mass of payload and sensors affect calculations
    # battery_mass_batt = 0.2 * (W_cruise/9.81)
    E_usable_batt = battery_mass_batt * specific_energy_batt * 0.8      # 0.8 for usable part of battery
    ratio_batt = E_usable_batt / E_mission_Wh * 100
    return battery_mass_batt, E_usable_batt, ratio_batt
```

and then is compared with the energy usage in the mission,

```python
def power_and_energy(W_cruise, S_cruise, A_hover, C_D_cruise,hover_frac_search):
    """
    Computes drag/thrust, induced velocities, segment powers, mission energy.
    Returns drag, induced (hover), P_cruise, P_hover, ascent/descent, energy in Wh.
    """
    D_cruise = q_cruise * S_cruise * C_D_cruise                    # Drag force, N
    v_induced_hover = math.sqrt(W_cruise / (2 * rho_cruise * A_hover)) # Induced hover velocity, m/s

    P_cruise = D_cruise * v_rel_avg_cruise / η_cruise
    P_hover = W_cruise * v_induced_hover / η_cruise
    P_ascent = 1.2 * P_hover
    P_descent = 0.9 * P_hover

    P_sensors_transport = 11.6
    P_sensors_search = 13.6

    t_cruise_travel = 20 * 60            # seconds
    t_cruise_search = 20 * 60            # seconds
    t_cruise = t_cruise_travel + t_cruise_search
    t_hover = 3 * 60
    t_ascent = 3 * 60
    t_descent = 3 * 60

    E_mission = (P_cruise * t_cruise + P_hover * t_hover +
                 P_ascent * t_ascent + P_descent * t_descent +
                 P_sensors_transport * t_cruise_travel + P_sensors_search * t_cruise_search +
                 P_hover * hover_frac_search * t_cruise_search)
    E_mission_Wh = E_mission / 3600 #conver to Wh

    return D_cruise, v_induced_hover, P_cruise, P_hover, P_ascent, P_descent, E_mission_Wh
```

(4) $$C_{airfoi_min} < C_L < C_{airfoil_max}$$

where S1223 airfoil are imported as an array,

```python
#S1223 airfoil data
alpha_array_cruise = np.array([
-7.25, -7.00, -6.75, -6.50, -6.25, -6.00, -5.75, -5.50, -5.25, -5.00, -4.75, -4.50, -4.25,
-3.75, -3.50, -3.25, -3.00, -2.75, -2.50, -2.25, -2.00, -1.75, -1.50, -1.25, -1.00, -0.75,
-0.50, -0.25, 0.00, 0.25, 0.50, 1.00, 1.25, 1.50, 1.75, 2.00, 2.25, 2.50, 2.75, 3.00, 3.25,
3.50, 3.75, 4.25, 4.50, 4.75, 5.00, 5.25, 5.50, 5.75, 6.00, 6.25, 6.50, 6.75, 7.00, 7.25,
7.50, 7.75, 8.00, 8.25, 8.50, 8.75, 9.00, 9.25, 9.50, 9.75, 10.00, 10.25, 10.50, 10.75,
11.00, 11.25, 11.50, 11.75, 12.00, 12.25, 12.50, 12.75, 13.00, 13.25, 13.50, 13.75, 14.00,
14.25, 14.50, 14.75, 15.00, 15.25, 15.50, 15.75, 16.00, 16.25, 16.50, 16.75, 17.00
])

cl_array_cruise = np.array([
0.1102, 0.1307, 0.1355, 0.2154, 0.2372, 0.1503, 0.1624, 0.1830, 0.2130,
0.2652, 0.3237, 0.3106, 0.3503, 0.3379, 0.3471, 0.3608, 0.4487, 0.4581,
0.4860, 0.8394, 0.8855, 0.9290, 0.9657, 0.9988, 1.0374, 1.0715, 1.1069,
1.1406, 1.1850, 1.2132, 1.2409, 1.2962, 1.3244, 1.3124, 1.3804, 1.4009,
1.4371, 1.4655, 1.4939, 1.5238, 1.5500, 1.5799, 1.6002, 1.6051, 1.6794,
1.7050, 1.7301, 1.7560, 1.7649, 1.8082, 1.8316, 1.8547, 1.8777, 1.9003,
1.9234, 1.9495, 1.9706, 1.9920, 2.0132, 2.0331, 2.0533, 2.0760, 2.0941,
2.1121, 2.1296, 2.1463, 2.1645, 2.1826, 2.2002, 2.2165, 2.2310, 2.2486,
2.2650, 2.2779, 2.2927, 2.3063, 2.3169, 2.3293, 2.3370, 2.3453, 2.3589,
2.3523, 2.3534, 2.3524, 2.3470, 2.3388, 2.3327, 2.3281, 2.3096, 2.2926,
2.2771, 2.2516, 2.2347, 2.2066, 2.1825
])

cd_array_cruise = np.array([
0.00424, 0.00489, 0.00605, 0.00577, 0.00715, 0.00859, 0.07791, 0.07507, 0.07167,
0.06686, 0.06171, 0.05955, 0.05680, 0.05285, 0.05148, 0.04886, 0.03815, 0.03829,
0.03691, 0.01585, 0.01490, 0.01439, 0.01417, 0.01418, 0.01397, 0.01396, 0.01398,
0.01398, 0.01374, 0.01392, 0.01419, 0.01448, 0.01464, 0.01484, 0.01511, 0.01530,
0.01544, 0.01561, 0.01579, 0.01599, 0.01624, 0.01659, 0.01676, 0.01710, 0.01699,
0.01726, 0.01755, 0.01790, 0.01837, 0.01859, 0.01885, 0.01912, 0.01941, 0.01972,
0.02011, 0.02059, 0.02087, 0.02118, 0.02151, 0.02185, 0.02227, 0.02284, 0.02317,
0.02352, 0.02392, 0.02444, 0.02503, 0.02543, 0.02590, 0.02647, 0.02722, 0.02773,
0.02831, 0.02912, 0.02986, 0.03066, 0.03171, 0.03266, 0.03399, 0.03532, 0.03692,
0.03894, 0.04107, 0.04349, 0.04644, 0.04985, 0.05322, 0.05751, 0.06179, 0.06710,
0.07270, 0.08001, 0.08627, 0.09454, 0.10230
])
```

then C_L is calculated using lift formula with the assumption of SLUF,

```python
#C_L_cruise calculated using the weight "felt" by the aircraft to account
    #for lift from vertical props in slow-speed cruise
C_L_cruise = (W_cruise - W_cruise*hover_frac_search) / (q_search * S_cruise)

#check if C_L_cruise go out of bounds
if C_L_cruise < np.min(cl_array_cruise) or C_L_cruise > np.max(cl_array_cruise):
    return None, None, None

AOA_cruise = np.interp(C_L_cruise, cl_array_cruise, alpha_array_cruise)
```

These constraints were then implemented in a nested for loop, referencing aforementioned functions for values.

Wing Sizing – Ranking Sims, Choosing Best

```python
################################################################################
# 7. Scoring and Selection

def score_design(d):
    # Higher is better, penalize excess weight, stall speed, loading, drag.
    score = (
        + 10 * d["ratio_batt"]
        # - 2 * (d["W_N_cruise"] / 9.81)
        - 1000 * (d["V_stall_cruise"] - v_rel_avg_cruise)**2
        - 5 * d["sigma_cruise"]
        - 5 * d["sigma_hover"]
        - 0.2 * d["D_cruise"]
        + 0.5 * d["LD_cruise"]
        - 100*d["S_cruise"]
    )
    return score

def find_best_design(results, n=10):
    if not results:
        print("No feasible designs found!")
        return None, []
    ranked = sorted(results, key=lambda d: score_design(d), reverse=True)
    return ranked[0], ranked[:n]
```

Paired with the constraints aforementioned, the output will *only* return viable options, sorted by most feasible designs. For each of these configurations, relevant data including MTOW (in newtons and pounds), W/S (wing loading ratio), D/L (disk loading ratio), S (wing area), A (total rotor area), b (wingspan), stall velocity, angle of attack, L/D ratio, excess energy %, and hover_frac are outputted by the following code:

```python
# 8. Output formatting

def print_row_aligned(d=None):
    """
    Output.
    """
    fields = [
        ("N_N", "N_N_cruise"),
        ("Ibs", "N_lbs_cruise"),
        ("W/S", "sigma_cruise"),
        ("D/L", "sigma_hover"),
        ("S", "S_cruise"),
        ("A", "A_hover"),
        ("B_rotors", "B_rotors_hover"),
        ("span_b", "b_cruise"),
        ("V_stall", "V_stall_cruise"),
        ("AOA", "AOA_cruise"),
        ("L/D", "LD_cruise"),
        ("%", "ratio_batt"),
        ("D", "D_cruise"),
        ("hover_frac_search", "hover_frac_search")
    ]
    w = 10
    sep = " "
    for label, key in fields:
        val = d[key]/lbs[]
        row += f"{label:{fmt(val)}{w}}"
    print(row)
```

```python
################################################################################
# 9. Main Algorithm

solutions = evaluate_designs(
    MTOW_range=MTOW_range,
    S_range=S_range,
    A_range=A_range,
    hover_frac_search_range=hover_frac_search_range
)
best, top10 = find_best_design(solutions)
print("BEST DESIGN:")
print_row_aligned(best)
print("\nTOP 10 CANDIDATES:")
for d in top10:
    print_row_aligned(d)
```

The final output with our best + top 10 design configuration specifications are as follows,

```
BEST DESIGN:
N_N: 860     Ibs: 193.34  W/S: 430.0   D/L: 26.67   S: 2     A: 30    B_rotors: 3.09   span_b: 3.46   V_stall: 6.36   AOA: 0.04   L/D: 9.04   %: 100.06   D: 38.54   hover_frac_search: 0.9

TOP 10 CANDIDATES:
N_N: 860     Ibs: 193.34  W/S: 430.0   D/L: 26.67   S: 2     A: 30    B_rotors: 3.09   span_b: 3.46   V_stall: 6.36   AOA: 0.04   L/D: 9.04   %: 100.06   D: 38.54   hover_frac_search: 0.9
N_N: 820     Ibs: 184.42  W/S: 410.0   D/L: 26.0    S: 2     A: 30    B_rotors: 3.09   span_b: 3.46   V_stall: 6.30   AOA: -0.13  L/D: 9.98   %: 100.43   D: 37.11   hover_frac_search: 0.9
N_N: 840     Ibs: 188.84  W/S: 420.0   D/L: 26.0    S: 2     A: 30    B_rotors: 3.09   span_b: 3.46   V_stall: 6.28   AOA: -0.13  L/D: 9.98   %: 100.22   D: 37.11   hover_frac_search: 0.9
N_N: 830     Ibs: 186.63  W/S: 410.0   D/L: 27.33   S: 2     A: 30    B_rotors: 3.09   span_b: 3.46   V_stall: 6.25   AOA: -0.1   L/D: 10.13  %: 100.45   D: 36.71   hover_frac_search: 0.9
N_N: 820     Ibs: 184.34  W/S: 410.0   D/L: 28.28   S: 2     A: 29    B_rotors: 3.04   span_b: 3.46   V_stall: 6.22   AOA: -0.3   L/D: 10.23  %: 101.03   D: 36.71   hover_frac_search: 0.9
N_N: 480     Ibs: 89.92   W/S: 200.0   D/L: 13.33   S: 2     A: 30    B_rotors: 3.09   span_b: 3.46   V_stall: 6.13   AOA: -0.5   L/D: 10.28  %: 102.74   D: 34.31   hover_frac_search: 0.9
N_N: 480     Ibs: 89.92   W/S: 200.0   D/L: 13.70   S: 2     A: 29    B_rotors: 3.04   span_b: 3.46   V_stall: 6.13   AOA: -0.5   L/D: 10.28  %: 101.88   D: 34.31   hover_frac_search: 0.9
N_N: 480     Ibs: 89.92   W/S: 200.0   D/L: 14.29   S: 2     A: 28    B_rotors: 2.99   span_b: 3.46   V_stall: 6.13   AOA: -0.5   L/D: 10.28  %: 100.96   D: 34.31   hover_frac_search: 0.9
N_N: 480     Ibs: 89.92   W/S: 200.0   D/L: 14.81   S: 2     A: 27    B_rotors: 2.93   span_b: 3.46   V_stall: 6.13   AOA: -0.5   L/D: 10.28  %: 100.0    D: 34.31   hover_frac_search: 0.9
N_N: 800     Ibs: 179.85  W/S: 400.0   D/L: 26.67   S: 2     A: 30    B_rotors: 3.09   span_b: 3.46   V_stall: 6.13   AOA: -0.5   L/D: 10.28  %: 101.00   D: 34.31   hover_frac_search: 0.9
```

The conventional values for Oswald efficiency number, e=0.8, and propeller efficiency, η=0.7, are used. Calculation of these values would require advanced CFD and/or wind tunnel testing, which do not fit within the scope of this challenge.

Final Design

BEST DESIGN:

W_N: 860	lbs: 193.34	W/S: 430.0	DL: 28.67	S: 2	A: 30	D_rotors: 3.09	span_b: 3.46	V_stall: 6.36	AOA: 0.04	L/D: 9.84	%: 100.06	D: 38.54	hover_frac_search: 0.9

TOP 10 CANDIDATES:

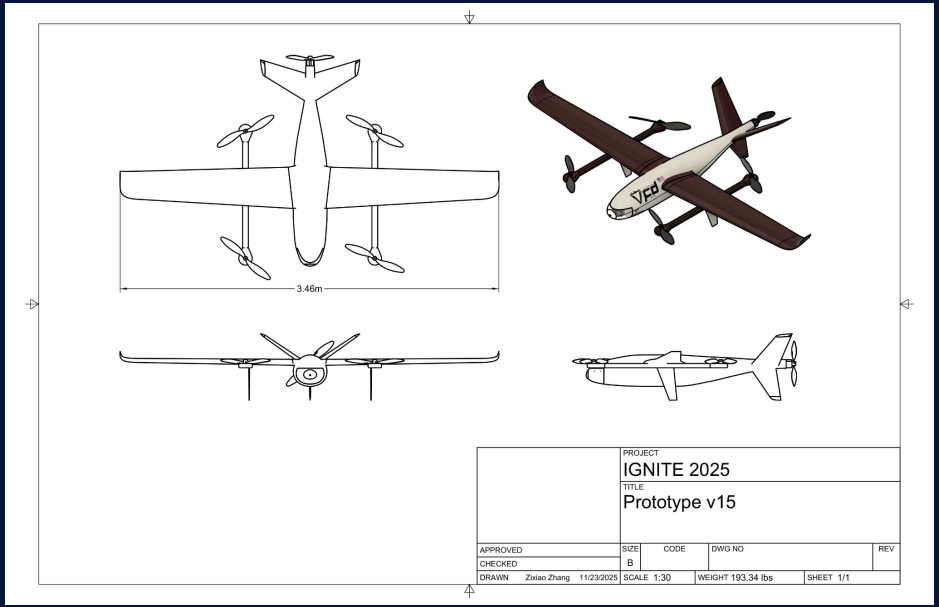
W_N: 860	lbs: 193.34	W/S: 430.0	DL: 28.67	S: 2	A: 30	D_rotors: 3.09	span_b: 3.46	V_stall: 6.36	AOA: 0.04	L/D: 9.84	%: 100.06	D: 38.54	hover_frac_search: 0.9
W_N: 420	lbs: 94.42	W/S: 210.0	DL: 14.0	S: 2	A: 30	D_rotors: 3.09	span_b: 3.46	V_stall: 6.28	AOA: -0.13	L/D: 9.98	%: 100.43	D: 37.11	hover_frac_search: 0.8
W_N: 840	lbs: 188.84	W/S: 420.0	DL: 28.0	S: 2	A: 30	D_rotors: 3.09	span_b: 3.46	V_stall: 6.28	AOA: -0.13	L/D: 9.98	%: 101.22	D: 37.11	hover_frac_search: 0.9
W_N: 820	lbs: 184.34	W/S: 410.0	DL: 27.33	S: 2	A: 30	D_rotors: 3.09	span_b: 3.46	V_stall: 6.21	AOA: -0.3	L/D: 10.13	%: 102.41	D: 35.71	hover_frac_search: 0.9
W_N: 820	lbs: 184.34	W/S: 410.0	DL: 28.28	S: 2	A: 29	D_rotors: 3.04	span_b: 3.46	V_stall: 6.21	AOA: -0.3	L/D: 10.13	%: 101.09	D: 35.71	hover_frac_search: 0.9
W_N: 400	lbs: 89.92	W/S: 200.0	DL: 13.33	S: 2	A: 30	D_rotors: 3.09	span_b: 3.46	V_stall: 6.13	AOA: -0.5	L/D: 10.28	%: 102.78	D: 34.31	hover_frac_search: 0.8
W_N: 400	lbs: 89.92	W/S: 200.0	DL: 13.79	S: 2	A: 29	D_rotors: 3.04	span_b: 3.46	V_stall: 6.13	AOA: -0.5	L/D: 10.28	%: 101.88	D: 34.31	hover_frac_search: 0.8
W_N: 400	lbs: 89.92	W/S: 200.0	DL: 14.29	S: 2	A: 28	D_rotors: 2.99	span_b: 3.46	V_stall: 6.13	AOA: -0.5	L/D: 10.28	%: 100.96	D: 34.31	hover_frac_search: 0.8
W_N: 400	lbs: 89.92	W/S: 200.0	DL: 14.81	S: 2	A: 27	D_rotors: 2.93	span_b: 3.46	V_stall: 6.13	AOA: -0.5	L/D: 10.28	%: 100.0	D: 34.31	hover_frac_search: 0.8
W_N: 800	lbs: 179.85	W/S: 400.0	DL: 26.67	S: 2	A: 30	D_rotors: 3.09	span_b: 3.46	V_stall: 6.13	AOA: -0.5	L/D: 10.28	%: 103.66	D: 34.31	hover_frac_search: 0.9

```
--- Best Design Summary ---
MTOW: 405 N (91.05 lbs)
Wing Area (S): 2 m²
Rotor Area (A): 30 m²
Wing Span (b): 3.46 m
Total Rotor Diameter: 3.09 m
Wing Loading: 202.5 N/m²
Disk Loading: 13.5 N/m²
Stall Speed: 6.17 m/s, Factor of Safety (FoS): 3.0430599675850893
Energy Margin (assumes 80% usage limit): 102.17%


--- Cruise Performance ---
Relative Velocity (v_rel_avg_cruise): 18.78 m/s
Lift Coefficient (C_L_cruise): 1.270
Angle of Attack (AOA_cruise): -0.40 deg
Drag Coefficient (C_D_cruise): 0.1098
Required Thrust (=D_cruise): 35.02 N
Induced Velocity (Cruise, v_induced_cruise): 0.803 m/s
```

```
--- Propeller Information -- Cruise ---
Number Props (N_cruise): 1
Propeller Diameter (D_cruise): 1.335 m
Disk Area (A_cruise): 1.400 m²
Shaft RPM (rpm_cruise): 402.1
Torque (Q_cruise): 26.714 N·m
Pitch (pitch_cruise): 2.801 m/rev
Tip Speed: 28.12 m/s (Mach 0.086)

--- Propeller Information -- Hover ---
Number Props (N_hover): 4
Prop Diameter (D_hover): 3.090 m
Suggested RPM (rpm_limit_hover): 1420.7
Torque per Prop (Torque_per_hover): 3.320 N·m
Tip Speed Actual: 229.88 m/s (Mach 0.700)
Disk Loading (Per Prop): 13.50 N/m²
```

```
--- Best Design Summary ---
MTOW: 860 N (193.34 lbs)
Wing Area (S): 2 m²
Rotor Area (A): 30 m²
Wing Span (b): 3.46 m
Total Rotor Diameter: 3.09 m
Wing Loading: 430.0 N/m²
Disk Loading: 28.67 N/m²
Stall Speed: 6.36 m/s, Factor of Safety (FoS): 2.9521509433962265
Energy Margin (assumes 80% usage limit): 100.06%


--- Cruise Performance ---
Relative Velocity (v_rel_avg_cruise): 18.78 m/s
Lift Coefficient (C_L_cruise): 2.697
Angle of Attack (AOA_cruise): 0.04 deg
Drag Coefficient (C_D_cruise): 0.1289
Required Thrust (=D_cruise): 38.56 N
Induced Velocity (Cruise, v_induced_cruise): 0.843 m/s

--- Propeller Information -- Cruise ---
Number Props (N_cruise): 1
Propeller Diameter (D_cruise): 1.401 m
Disk Area (A_cruise): 1.542 m²
Shaft RPM (rpm_cruise): 383.3
Torque (Q_cruise): 30.854 N·m
Pitch (pitch_cruise): 2.939 m/rev
Tip Speed: 28.12 m/s (Mach 0.086)

--- Propeller Information -- Hover ---
Number Props (N_hover): 4
Prop Diameter (D_hover): 3.090 m
Suggested RPM (rpm_limit_hover): 1420.7
Torque per Prop (Torque_per_hover): 10.272 N·m
Tip Speed Actual: 229.88 m/s (Mach 0.700)
Disk Loading (Per Prop): 28.67 N/m²

--- MISC ---
Mean Aerodynamic Chord (chord_cruise): 0.5773502691896257 m
Reynolds number (Re_cruise): 548028.824170266
```

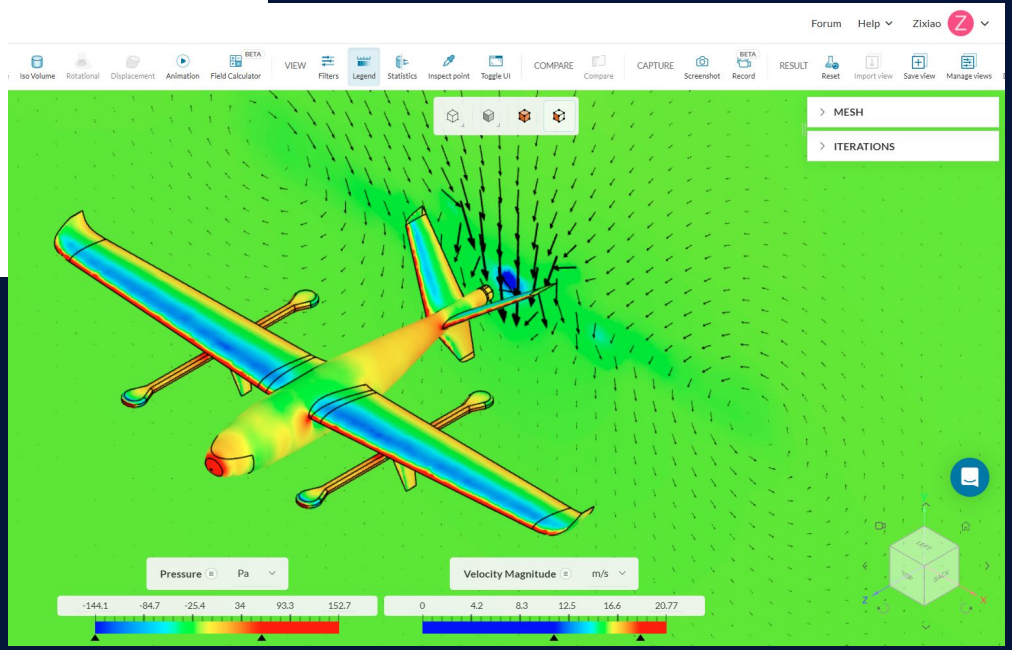
Wing sizing


3.46m
PROJECT
IGNITE 2025
TITLE
Prototype v15
APPROVED
CHECKED
DRAWN Zixiao Zhang 11/23/2025
SIZE B
CODE
DWG NO
REV
SCALE 1:30
WEIGHT 193.34 lbs
SHEET 1/1

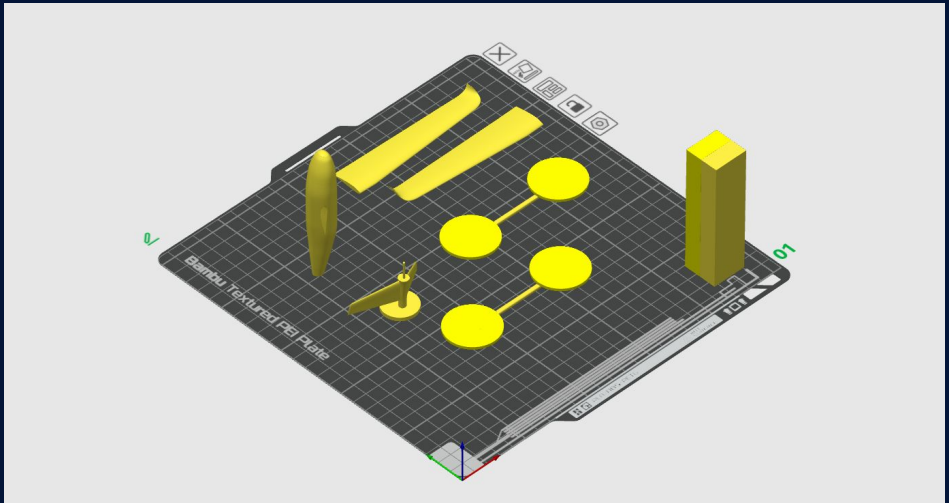
Stability

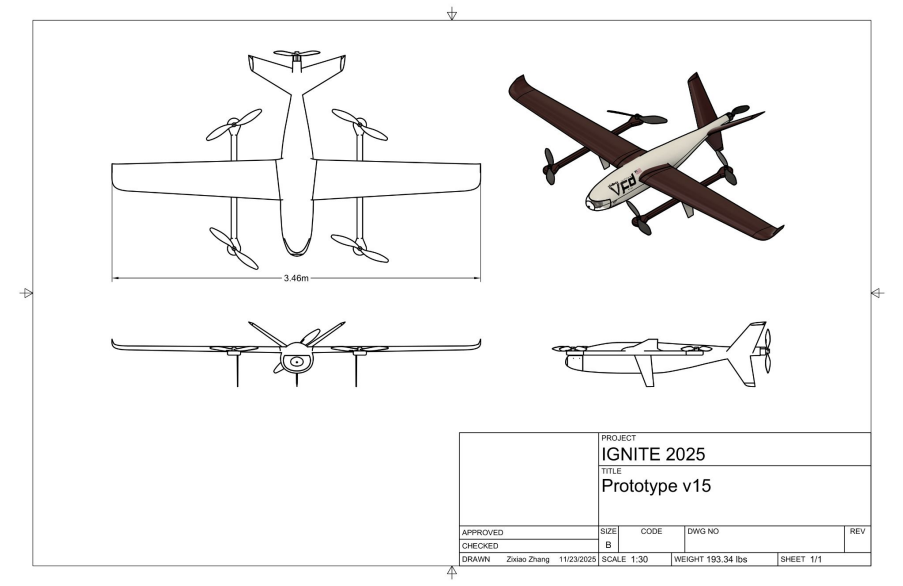
N867CA

CAD/CFD


Forum
Help
Zixiao
Iso Volume
Rotational
Displacement
Animation
Field Calculator
BETA
VIEW
Filters
Legend
Statistics
Inspect point
Toggle UI
COMPARE
Compare
CAPTURE
Screenshot
Record
RESULT
Reset
Import view
Save view
Manage views
MESH
ITERATIONS
Pressure
Pa
-144.1
-84.7
-25.4
34
93.3
152.7
Velocity Magnitude
m/s
0
4.2
8.3
12.5
16.6
20.77

FRONT


Bambu Textured PEI Plate


3.46m
PROJECT
IGNITE 2025
TITLE
Prototype v15
APPROVED
CHECKED
DRAWN Zixiao Zhang 11/23/2025
SIZE B
CODE
DWG NO
REV
SCALE 1:30
WEIGHT 193.34 lbs
SHEET 1/1

Questions?